

Redundância e Desacoplamento de Serviços

Infraestruturas de Computação na Nuvem, Aula 09

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda (1/2)

- Redundância ativa-ativa vs ativa-passiva.
- Single points of failure.
- Métricas de recuperação: RTO e RPO.
- Motivação para o desacoplamento entre serviços.
- Comunicação síncrona vs assíncrona.

Agenda (2/2)

- Filas de mensagens e brokers: RabbitMQ e AMQP.
- Componentes AMQP: exchanges, bindings e filas.
- Garantias de entrega e idempotência.
- Padrões de mensageria e de resiliência.
- Message brokers vs event streaming.
- Redundância e desacoplamento no Kubernetes.

Redundância Ativa-Ativa vs Ativa-Passiva

- Redundância: manter mais do que uma instância de um componente crítico.
- **Ativa-ativa**: todas as réplicas processam pedidos em simultâneo.
- **Ativa-passiva**: uma réplica processa pedidos, outra(s) ficam em espera (*standby*) para assumir em caso de falha.
- Ativa-ativa aproveita melhor os recursos; ativa-passiva é mais simples de garantir consistência.

Single Points of Failure

- **Single point of failure (SPOF)**: qualquer componente cuja falha isolada derruba todo o serviço.
- Pode ser um servidor, um load balancer, uma base de dados ou até um único link de rede.
- Estratégia: identificar SPOFs e introduzir redundância em cada um.
- Redundância sem eliminar todos os SPOFs dá uma falsa sensação de segurança.

Métricas de Recuperação: RTO e RPO

- **RTO** (Recovery Time Objective): quanto tempo o serviço pode estar indisponível antes de o impacto ser inaceitável.
- **RPO** (Recovery Point Objective): quantos dados se pode perder, medido em tempo desde o último ponto de recuperação.
- Um RPO de 15 minutos implica replicação ou backups com, no máximo, esse intervalo.
- Uma configuração ativa-ativa tende a dar RTO próximo de zero; restaurar a partir de backup dá um RTO de horas.
- Estes valores são decisões de negócio, com custo associado: quanto mais exigentes, mais cara a infraestrutura necessária.

Porque Desacoplar Serviços

- Serviços fortemente acoplados propagam falhas em cascata: um lento, todos lentos.
- Desacoplamento permite que cada serviço evolua, seja atualizado e escale de forma independente.
- Reduz o impacto de uma falha isolada no sistema global.
- Base para arquiteturas de microsserviços resilientes.

Comunicação Síncrona vs Assíncrona

- **Síncrona** (ex. chamadas REST diretas): o cliente espera pela resposta antes de continuar.
- Simples de entender, mas acopla a disponibilidade do cliente à do serviço chamado.
- **Assíncrona** (baseada em mensagens): o remetente envia e continua, sem esperar resposta imediata.
- Tolerar melhor picos de carga e indisponibilidades temporárias do lado receptor.

Filas de Mensagens e Brokers

- Um **broker de mensagens** recebe, guarda e entrega mensagens entre produtores e consumidores.
- Produtor e consumidor não precisam de estar ativos ao mesmo tempo – desacoplamento temporal.
- **RabbitMQ**: broker de mensagens amplamente usado, baseado no protocolo **AMQP**.
- AMQP define exchanges, filas e regras de encaminhamento entre eles.

Componentes do AMQP

- O produtor *nunca* publica diretamente numa fila: publica sempre num **exchange**.
- O **binding** liga um exchange a uma fila, eventualmente com uma *routing key* que atua como critério de filtragem.
- **Direct**: entrega às filas cuja routing key corresponde exatamente à da mensagem.
- **Fanout**: ignora a routing key e entrega a todas as filas ligadas – é a base do publish/subscribe.
- **Topic**: correspondência por padrões com *wildcards* (ex. logs.*.erro).
- **Headers**: encaminha com base em cabeçalhos da mensagem, em vez da routing key.

Garantias de Entrega

- **At-most-once**: a mensagem pode perder-se, mas nunca é processada duas vezes.
- **At-least-once**: a mensagem nunca se perde, mas pode ser entregue mais do que uma vez.
- **Exactly-once**: entregue exatamente uma vez – difícil e dispendioso de garantir em sistemas distribuídos.
- Na prática, a solução comum é *at-least-once* com consumidores **idempotentes**: processar duas vezes a mesma mensagem não altera o resultado.
- **Acknowledgements (ACK)**: o consumidor confirma o processamento; sem confirmação, o broker volta a entregar a mensagem.
- **Durabilidade**: filas e mensagens marcadas como persistentes sobrevivem ao

Padrões de Mensageria

- **Work queues:** várias tarefas distribuídas por múltiplos consumidores, cada mensagem processada uma única vez.
- Útil para distribuir trabalho pesado por vários workers em paralelo.
- **Publish/subscribe:** uma mensagem é difundida para todos os consumidores subscritos.
- Útil quando vários serviços independentes precisam de reagir ao mesmo evento.

Padrões de Resiliência

- **Circuit breaker:** deteta falhas repetidas numa chamada a um serviço e passa a bloquear novas tentativas temporariamente.
- Evita sobrecarregar um serviço já em dificuldades e dá-lhe tempo para recuperar.
- **Retries com backoff exponencial:** repetir uma operação falhada, aumentando o intervalo entre tentativas.
- Reduz a pressão sobre o sistema em falha, em vez de o inundar com pedidos repetidos imediatos.

Filas de Mensagens Mortas e Contrapressão

- **Dead letter queue (DLQ)**: fila para onde são desviadas mensagens que não puderam ser processadas com sucesso.
- Evita que uma mensagem problemática bloqueie indefinidamente o processamento das restantes (*poison message*).
- **Backpressure**: quando os consumidores não acompanham o ritmo dos produtores, a fila cresce sem limite.
- Mitigação: limitar o tamanho das filas, aplicar *prefetch* nos consumidores, ou escalar o número de consumidores.
- O tamanho da fila é, por isso, uma excelente métrica para auto-scaling – como referido na Aula 08.

Message Brokers vs Event Streaming

- **Message broker** (ex. RabbitMQ): a mensagem é removida da fila após ser consumida e confirmada.
- Orientado a *tarefas*: cada mensagem representa trabalho a executar uma vez.
- **Event streaming** (ex. Apache Kafka): os eventos são guardados num registo ordenado e persistente (*log*).
- Vários consumidores leem independentemente, cada um com o seu *offset*; os eventos permanecem disponíveis para releitura.
- Orientado a *eventos*: adequado a análise, auditoria e reprocessamento histórico.
- A escolha depende do caso de uso: distribuir trabalho (broker) ou registar factos (streaming).

Redundância e Desacoplamento no Kubernetes

- Várias réplicas de um Deployment atrás de um Service constituem redundância **ativa-ativa** (Aula 08).
- **PodDisruptionBudget**: garante um número mínimo de réplicas disponíveis durante operações de manutenção planeada.
- **Anti-affinity**: distribui as réplicas por nós diferentes, evitando que a falha de um nó derrube todas.
- Serviços com estado, como um broker de mensagens, implantam-se tipicamente com **StatefulSet** e armazenamento persistente.
- O desacoplamento por mensageria mantém-se igualmente válido dentro do cluster: produtores e consumidores são Deployments independentes.

Síntese da Aula (1/2)

- Redundância ativa-ativa ou ativa-passiva ajuda a eliminar single points of failure.
- Desacoplar serviços evita falhas em cascata e permite evolução independente.
- Comunicação assíncrona, via filas de mensagens, reduz o acoplamento temporal entre serviços.

Síntese da Aula (2/2)

- No AMQP, os produtores publicam em exchanges (direct, fanout, topic, headers), que encaminham para filas através de bindings.
- Work queues distribuem tarefas; publish/subscribe difunde eventos. ACKs e durabilidade sustentam a entrega *at-least-once* com consumidores idempotentes.
- DLQs isolam mensagens problemáticas; o tamanho da fila é uma boa métrica de auto-scaling.
- Circuit breaker e retries com backoff exponencial tornam os sistemas mais resilientes; RTO e RPO quantificam os objetivos de recuperação.

Obrigada.

Questões e comentários